Nodebook — Project Specification

Oct 31, 2026 · @Karo

1. Project overview

Nodebook is a web application that turns a user's daily notes into a personal knowledge graph: it proposes links between notes, the user approves or rejects them, and a query agent answers questions about the graph with citations to the source notes.

Problem. Notes written on different days are rarely revisited, so related ideas never meet. Existing tools either require links to be made by hand or offer AI search without an explicit, inspectable structure.

Goal. Deliver one complete loop within the semester: *write* a note, *connect* it to existing knowledge with user-reviewed suggestions, and *ask* questions answered from the user's own notes.

Target users. Students, researchers and knowledge workers, roughly 18–40, who keep regular notes. Two illustrative personas guide design decisions:

Persona	Context	Main need
Anna, 26	PhD student, daily lab notes and paper summaries	See how findings across months relate before writing
Michał, 34	Software engineer, learning journal	Ask "what do I already know about X?" in plain language

Project type. Web application: browser-based, responsive, no installation. A large screen suits the graph view and chat panel.

Scope. In scope: single-user private knowledge graphs, text notes, link suggestions, graph view, question answering. Out of scope: real-time collaboration, shared graphs, native mobile apps, handwriting or audio input.

Team. Five members; roles and ownership are listed under Deliverables.

2. Required core features

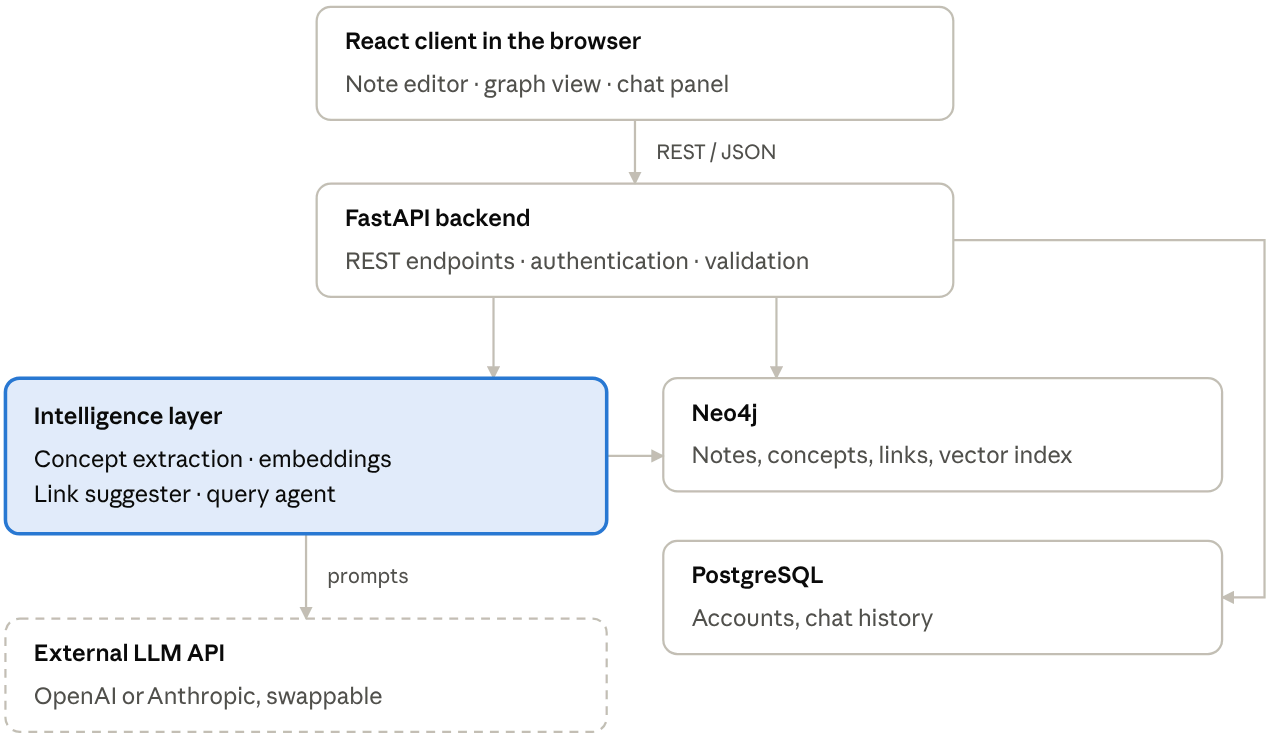
Six high-priority features (must have) form the minimum viable product; the project is not complete without all of them.

ID	Requirement	Acceptance criteria
H1	The system should allow users to register and log in with email and password.	Invalid email or password under 8 characters is rejected; passwords are stored hashed; sessions expire after inactivity; a user can log out.
H2	Users should be able to create, edit and delete dated text notes.	A note stores text, entry date and timestamps; edits are saved; deleting a note also removes its links.
H3	The system should suggest links between a saved note and existing notes.	After saving, up to 5 candidate links appear, each with a similarity score and a one-line reason; notes of other users are never suggested.
H4	Users should be able to accept or reject each suggested link.	Accepted links appear in the graph; rejected links are kept as rejected and not suggested again for the same pair.
H5	The system should display notes and accepted links as an interactive graph.	Nodes can be zoomed, panned and clicked to open the note; the view remains usable with 500 notes.
H6	Users should be able to ask questions about their notes and get cited answers.	Each answer lists the notes it used, with links to them; if no relevant note exists, the agent says so instead of answering from general knowledge.

3. Technical requirements

Nodebook is a layered web application with a Python backend, a TypeScript frontend and Neo4j as the graph store; the LLM provider sits behind the API so it can be replaced.

The intelligence layer sits behind the API, so the LLM can be swapped



Nodebook architecture · 6 components

The client talks only to the API; the query agent reads Neo4j directly through graph and vector-search tools, and only the intelligence layer calls the external LLM.

Technology stack (initial choices, may change after prototyping):

Layer	Technologies
Frontend	TypeScript, React, Vite, Tailwind CSS, Cytoscape.js (graph view), TanStack Query
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy + Alembic, JWT authentication
AI layer	LLM API (OpenAI or Anthropic), sentence-transformers embeddings, LangGraph tool-using agent
Data	Neo4j 5 (notes, concepts, links, vector index), PostgreSQL (accounts, chat history)

Layer	Technologies
Testing	pytest, Vitest, Playwright (end-to-end), Ruff, ESLint
DevOps	Git + GitHub, GitHub Actions CI, Docker Compose, GitHub Projects (backlog)

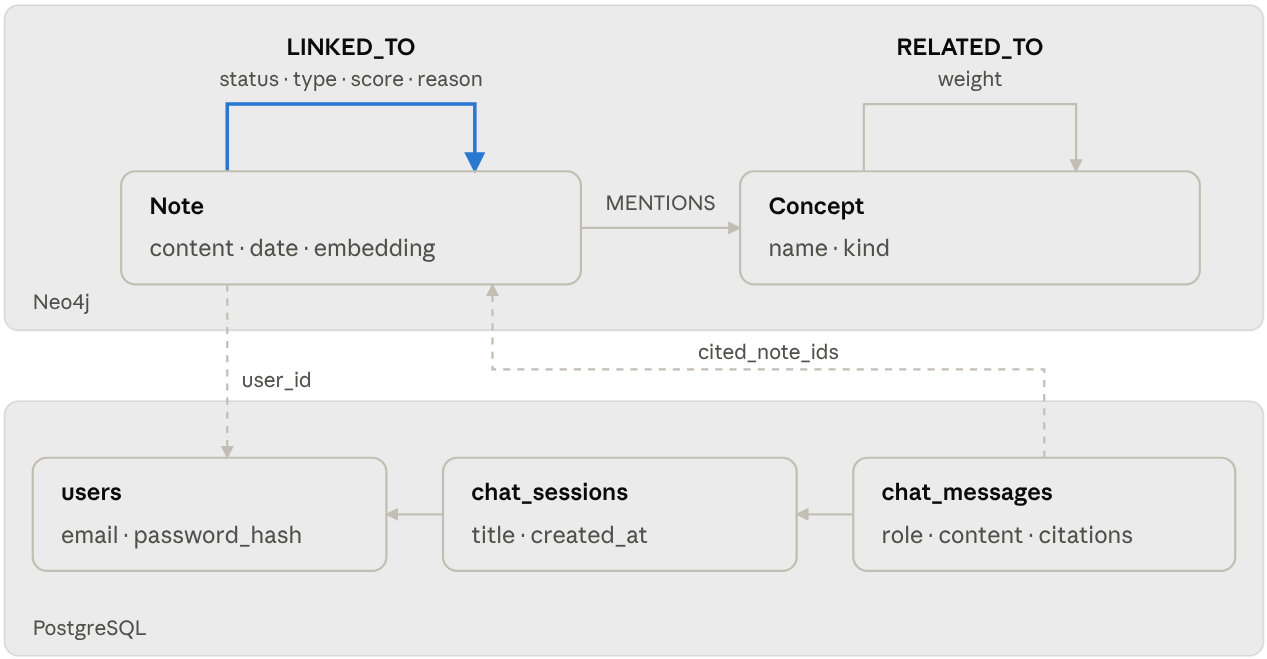
Non-functional requirements (targets for the MVP):

Area	Requirement
Performance	CRUD endpoints respond in under 500 ms at the 95th percentile; link suggestions appear within 10 s of saving a note; agent answers within 20 s.
Security	Passwords hashed with Argon2 or bcrypt; HTTPS only; API keys kept in environment variables, never in the repository.
Privacy	Every graph query is filtered by user id; a user can export and permanently delete all their data.
Reliability	LLM failures do not block saving notes; suggestions are retried in the background.
Usability	Responsive layout from 1280 px desktop down to tablet; core flows usable with keyboard only.
Maintainability	REST API documented through FastAPI's generated OpenAPI schema; code formatted and linted in CI.
Testing	At least 70% line coverage on the backend; an end-to-end test for the write, connect, ask loop.
Deployment	The whole system starts locally with a single `docker compose up`.

4. Suggested data model

Notes and concepts are graph nodes in Neo4j; a link between two notes is a relationship whose `status` records the user's decision, so suggested, accepted and rejected links share one structure. Accounts and chat history stay in PostgreSQL.

Every note-to-note link is one relationship whose status records the decision



Data model · 2 graph nodes, 3 relationships, 3 tables

Dashed arrows cross stores by id only: graph nodes carry `user_id`, and chat messages cite notes by `cited_note_ids`.

Graph nodes (Neo4j)

Label	Properties
`Note`	`id` (UUID), `user_id`, `content` (text), `entry_date` (date), `created_at`, `updated_at`, `embedding` (vector, indexed)
`Concept`	`id` (UUID), `user_id`, `name`, `normalized_name` (unique per user), `kind` (person, method, topic, place, other)

Graph relationships (Neo4j)

Relationship	Properties
`(Note)-[:LINKED_TO]->(Note)`	`status` (suggested, accepted, rejected), `type` (related, supports, contradicts, extends, example_of), `score` (0–1), `reason` (text), `origin` (system, user), `created_at`, `decided_at`
`(Note)-[:MENTIONS]->(Concept)`	`confidence` (0–1), `created_at`
`(Concept)-[:RELATED_TO]->(Concept)`	`weight` (number of notes in which both concepts appear)

Relational tables (PostgreSQL)

Table	Columns
`users`	`id` (UUID, PK), `email` (unique), `password_hash`, `display_name`, `created_at`
`chat_sessions`	`id` (PK), `user_id` (FK → users), `title`, `created_at`
`chat_messages`	`id` (PK), `session_id` (FK → chat_sessions), `role` (user, assistant), `content`, `cited_note_ids` (UUID array), `created_at`

Every Neo4j node carries `user_id`, and every query filters on it; this enforces per-user isolation without a separate database per user. A uniqueness constraint on `(user_id, normalized_name)` prevents duplicate concepts.

5. Optional extensions

Medium-priority extensions (should have) are planned once H1–H6 pass their acceptance criteria; low-priority ones (could have) are attempted only if time remains.

ID	Extension	Priority
M1	Users should be able to search notes by keyword and by meaning (semantic search).	Medium

ID	Extension	Priority
M2	The system should extract concepts from notes and show them as graph nodes.	Medium
M3	Users should be able to filter the graph by date range or concept.	Medium
M4	Users should be able to create and delete links manually.	Medium
M5	The system should send an optional daily writing reminder.	Medium
L1	Users should be able to import notes from Markdown files.	Low
L2	Users should be able to export notes and the graph as Markdown or JSON.	Low
L3	The system should generate a weekly digest of recurring themes.	Low
L4	The system should use rejected links to improve future suggestions.	Low
L5	The interface should offer dark mode and a mobile-responsive layout.	Low

M2 is a prerequisite for M3′s concept filter and for the `Concept` part of the data model; if it is dropped, the model reduces to notes and links.

6. Deliverables

The project ends with a deployed, tested MVP covering H1–H6, its source repository and a final presentation; the course deadlines below still need to be filled in.

#	Deliverable	Contents	Due
D1	Presentation 1: initial idea	Problem, target users, comparison with existing apps, project type, prioritised functionalities, technology stack	Delivered, Oct 2026

#	Deliverable	Contents	Due
D2	Project specification	This document, agreed by the team	[date]
D3	Design documentation	Use-case diagram, data model, component diagram, sequence diagrams for save-and-suggest and ask-the-agent, UI wireframes	[date]
D4	Working prototype	H1 and H2 end to end; a first version of H3	[date]
D5	MVP release	H1–H6 passing their acceptance criteria; selected medium-priority extensions	[date]
D6	Source code repository	GitHub repository with README, setup instructions, Docker Compose file, CI pipeline, tests	Continuous, final at D5
D7	Test report	Unit, integration and end-to-end results; coverage report; known issues	[date]
D8	Final presentation and demo	Live demo of the write, connect, ask loop; lessons learned	[date]

Team roles

Role	Responsibilities	Member
Project manager	Backlog, sprint planning, documentation, course deliverables	[name]
Frontend developer	React app, note editor, graph view, chat panel	[name]
Backend developer	FastAPI services, authentication, PostgreSQL	[name]
AI engineer	Concept extraction, embeddings, link suggestions, query agent	[name]
Data and QA engineer	Neo4j schema, test suite, CI, Docker deployment	[name]

Open question: which deliverables are graded, and are D3 diagrams required in a specific notation (UML 2.5)?